Universidad Tecnológica de Chihuahua

Desarrollo y Gestión de Software

APLICACIONES WEB PROGRESIVAS

ING. Alfonso Barroso

LABORATORIO DE PRUEBAS: PWA CONECTADA A UN BACKEND Y BASE DE DATOS

IDGS101N

Integrantes:

Aldo Alberto Barrera González
María José Durán Ramírez
Raúl Adrián Cano García
Oliver Manríquez Coronado
Ailton Márquez Vargas
Iram Guillermo Álvarez Fuyivara

Chihuahua, Chih., 14 de octubre de 2026

Índice

Introducción **3**
Desarrollo **4**
1. Descripción del laboratorio 4
2. Evidencias de persistencia y base de datos 4
2.1. Scripts de carga y limpieza 4
2.2. Estado inicial de la base de datos (antes del envío) 5
2.3. Consulta de información (GET) 6
2.4. Envío de una nueva transacción (POST) 7
2.5. Estado de la base de datos (después del envío) 7
2.6. Idempotencia del envío 8
2.7. Validación e integridad de datos 9
2.8. Logs del servidor Spring Boot 10
2.9. Métricas de memoria del servidor 11
3. Evidencias del frontend y comportamiento PWA 12
3.1. Listado de datos renderizado 12
3.2. DevTools: pestaña Network (consulta) 12
3.3. DevTools: pestaña Network (envío) 13
3.4. DevTools: Service Worker activo 14
3.5. DevTools: Cache Storage 15
3.6. DevTools: IndexedDB 16
3.7. Comportamiento sin conexión 17
3.8. Sincronización al recuperar la conexión 18
3.9. DevTools: pestaña Console 19
4. Resumen de casos de prueba 19
Conclusiones **21**
Referencias **22**

Introducción

El presente documento reúne las evidencias del laboratorio de pruebas de la aplicación web progresiva (PWA) del proyecto integrador Vestigia. El objetivo es validar la integración de la PWA con un backend de servicios y una base de datos: la ejecución correcta de las consultas, la renderización de los datos en la interfaz, la persistencia de nuevas transacciones y el comportamiento de la aplicación sin conexión.

Para esta primera etapa el laboratorio se limita a la entidad de salas, que permite comprobar de punta a punta los flujos de consulta (GET) y envío (POST) sin depender del diseño completo del modelo de datos.

Desarrollo

1. Descripción del laboratorio

La PWA, desarrollada con React, Vite y TypeScript, consume una API REST construida con Spring Boot que persiste la información en una base de datos PostgreSQL. La API expone el recurso de salas (/api/salas) y registra cada petición con su método, ruta, código de respuesta, tiempo de procesamiento y memoria utilizada. El envío de nuevas salas es idempotente por identificador, de modo que la PWA puede reenviar sin duplicar las operaciones que quedaron en cola mientras no había conexión.

2. Evidencias de persistencia y base de datos

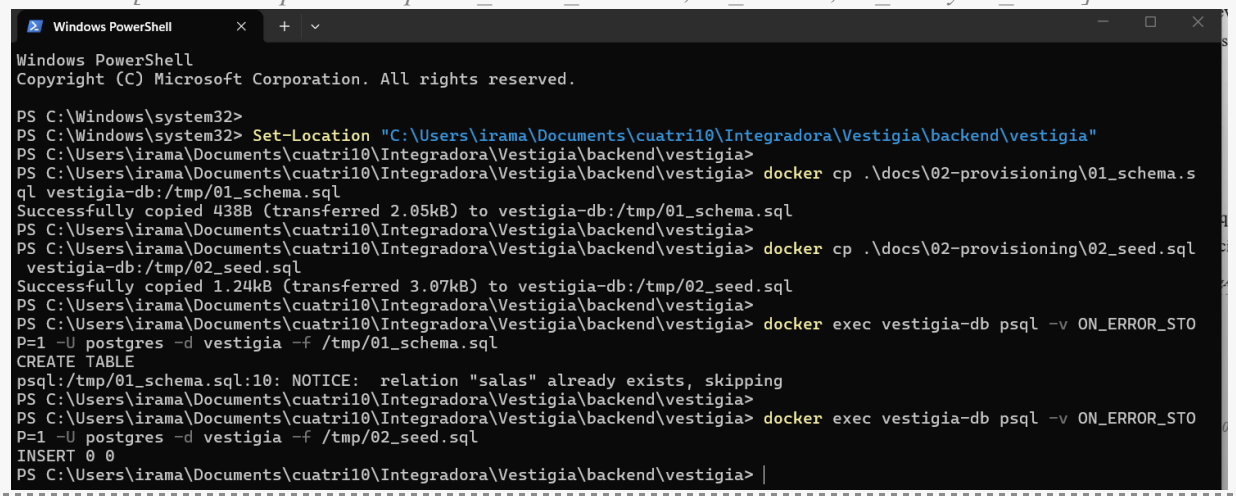
2.1. Scripts de carga y limpieza

Se utilizaron scripts SQL para crear la base de datos, el esquema y los datos iniciales, así como para limpiar la tabla y dejar el laboratorio en un estado conocido antes de cada prueba.

La base fue creada mediante POSTGRES_DB=vestigia y cambia “pgAdmin” por “psql en Docker”.

Figura 1. Scripts de provisioning ejecutados en pgAdmin

[Insertar captura: scripts 00_create_database, 01_schema, 02_seed y 99_clean]



2.2. Estado inicial de la base de datos (antes del envío)

Consulta a la tabla de salas antes de registrar nuevos datos, para establecer el punto de comparación con la estructura e integridad esperadas.

Figura 2. Resultado de SELECT sobre la tabla salas antes del envío

*[Insertar captura: SELECT * FROM salas; en pgAdmin]*

```
PS C:\Users\irama\Documents\cuatri10\Integradora\Vestigia\backend\vestigia> docker exec vestigia
-db psql -U postgres -d vestigia -c "SELECT COUNT(*) AS total FROM salas;"
 total
-------
     6
(1 row)

PS C:\Users\irama\Documents\cuatri10\Integradora\Vestigia\backend\vestigia> docker exec vestigia
-db psql -P pager=off -U postgres -d vestigia -c "SELECT id, nombre, escenario, activa FROM sala
s ORDER BY creada_en;"
                  id                  |        nombre         |  escenario   | activa
--------------------------------------+-----------------------+--------------+--------
 c1b7a1de-5f0e-4a58-9a3e-0b5a7f6d1a01 | Puesto del pescador   | pueblo       | t
 c1b7a1de-5f0e-4a58-9a3e-0b5a7f6d1a02 | Plaza del pueblo      | pueblo       | t
 c1b7a1de-5f0e-4a58-9a3e-0b5a7f6d1a03 | Base del faro         | faro         | t
 c1b7a1de-5f0e-4a58-9a3e-0b5a7f6d1a04 | Escalera del faro     | faro         | t
 c1b7a1de-5f0e-4a58-9a3e-0b5a7f6d1a05 | Linterna del faro     | faro         | t
 c1b7a1de-5f0e-4a58-9a3e-0b5a7f6d1a06 | Cúpula del telescopio | observatorio | t
(6 rows)

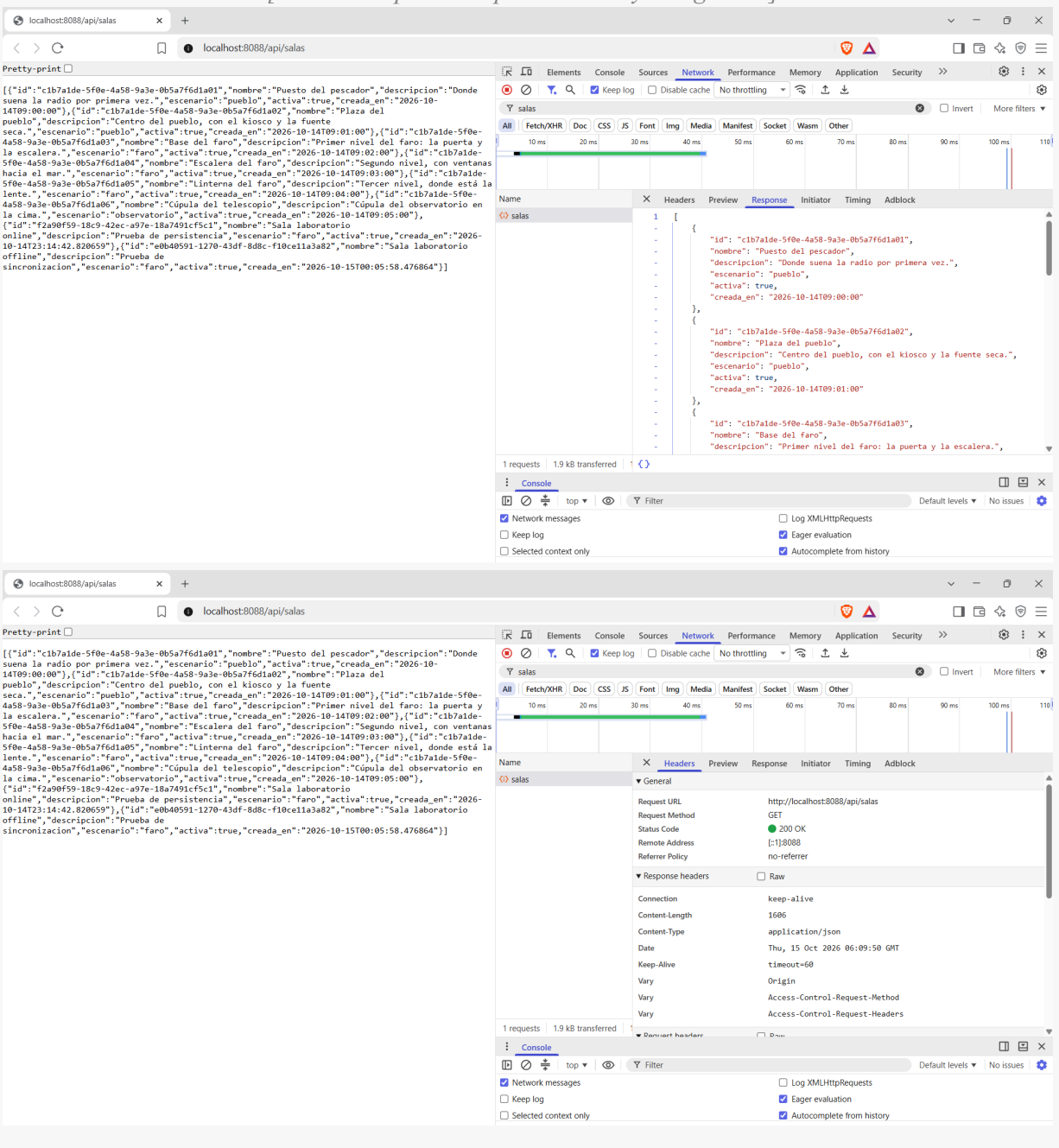
PS C:\Users\irama\Documents\cuatri10\Integradora\Vestigia\backend\vestigia>
```

2.3. Consulta de información (GET)

Respuesta de la API a la consulta del listado de salas, con código 200 y los datos en formato JSON.

Figura 3. Respuesta de GET /api/salas

[Insertar captura: respuesta JSON y código 200]

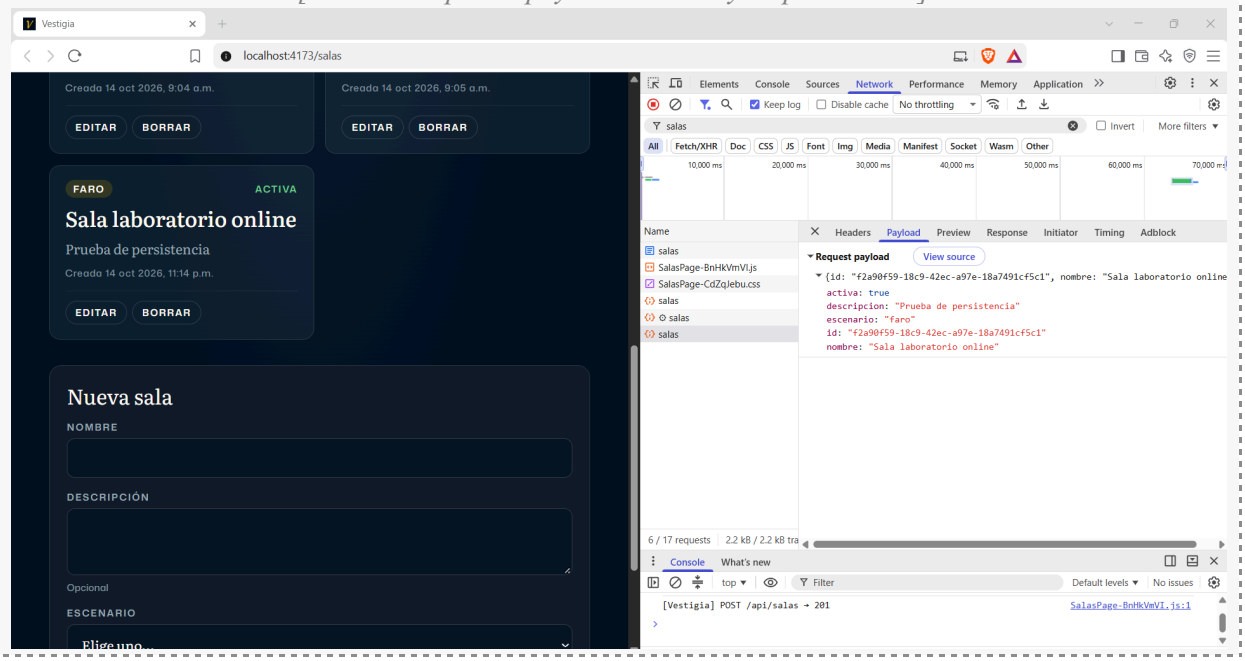


2.4. Envío de una nueva transacción (POST)

Envío de una nueva sala a la API y respuesta de creación (201) con el recurso generado.

Figura 4. Petición y respuesta de POST /api/salas

[Insertar captura: payload enviado y respuesta 201]

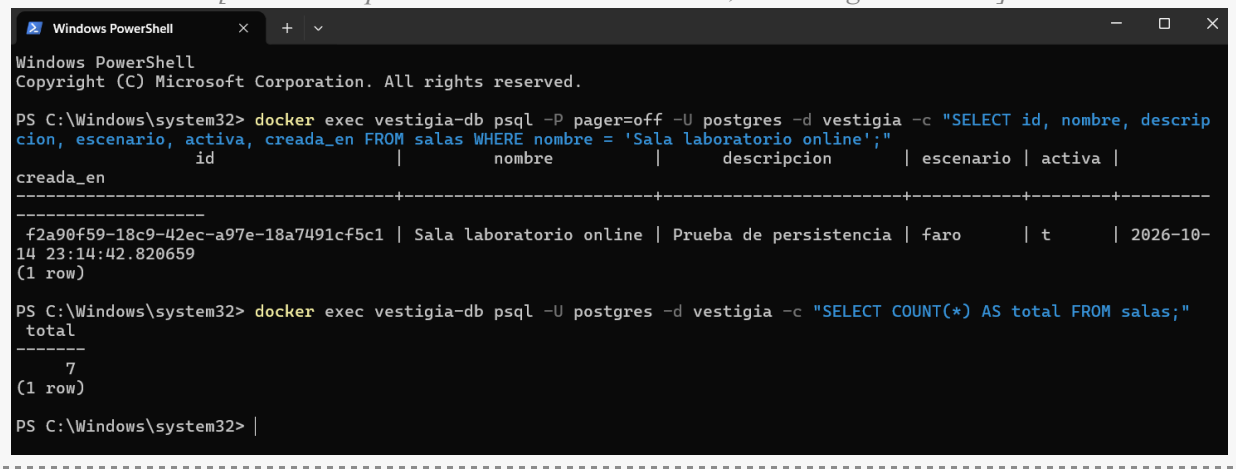


2.5. Estado de la base de datos (después del envío)

Consulta a la tabla de salas tras el envío, donde se verifica que el registro se almacenó con la estructura e integridad requeridas.

Figura 5. Resultado de SELECT sobre la tabla salas después del envío

*[Insertar captura: SELECT * FROM salas; con el registro nuevo]*

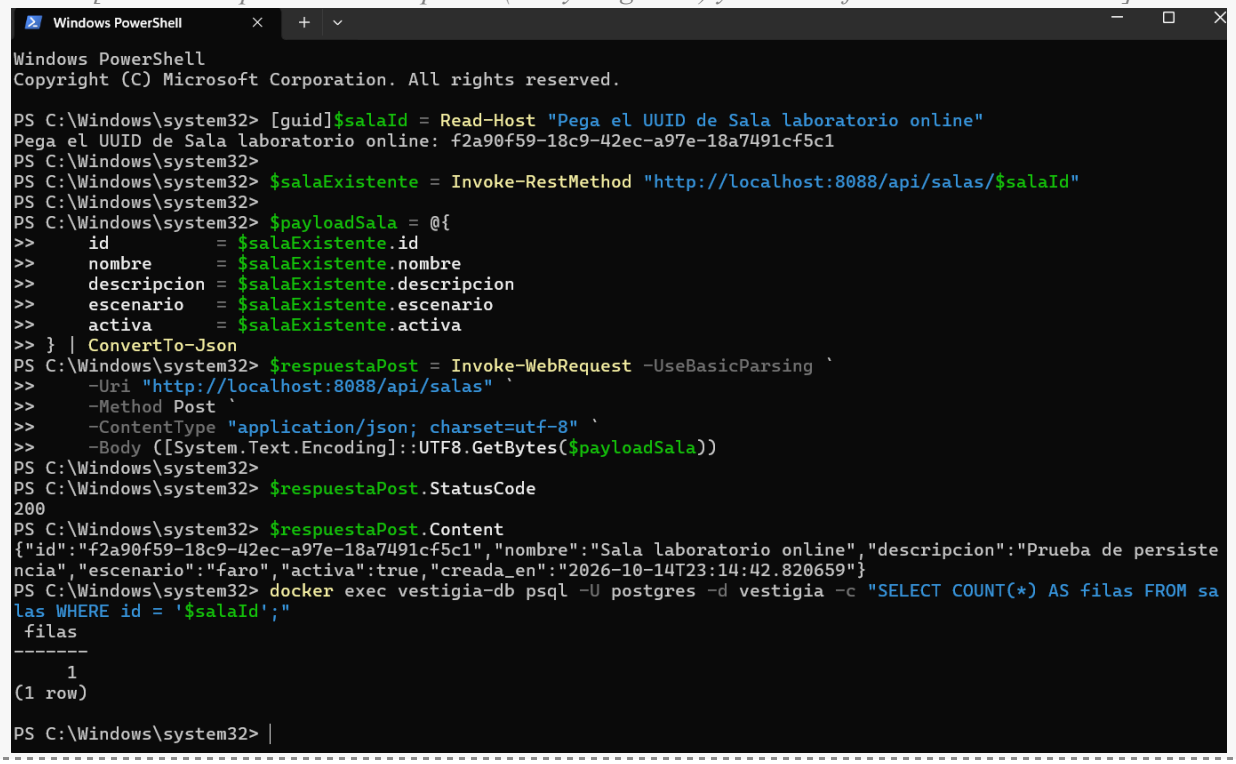


2.6. Idempotencia del envío

Reenvío de la misma sala con el mismo identificador. La API responde con código 200 sin modificar ni duplicar el registro, lo que permite sincronizar de forma segura la cola de operaciones pendientes de la PWA.

Figura 6. Reenvío del mismo identificador y consulta de la tabla

[Insertar captura: POST repetido (201 y luego 200) y una sola fila en la base de datos]

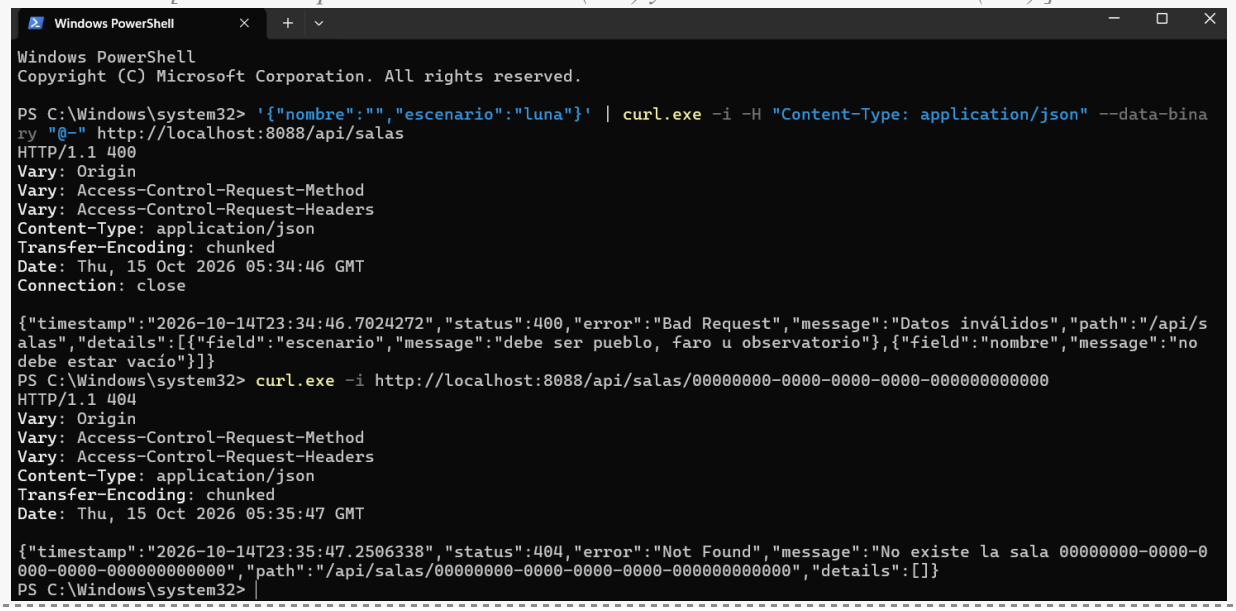


2.7. Validación e integridad de datos

Pruebas con datos inválidos y con un identificador inexistente, para comprobar que la API rechaza la información incorrecta y responde con un cuerpo de error uniforme.

Figura 7. Respuestas 400 y 404 de la API

[Insertar captura: POST inválido (400) y consulta de id inexistente (404)]

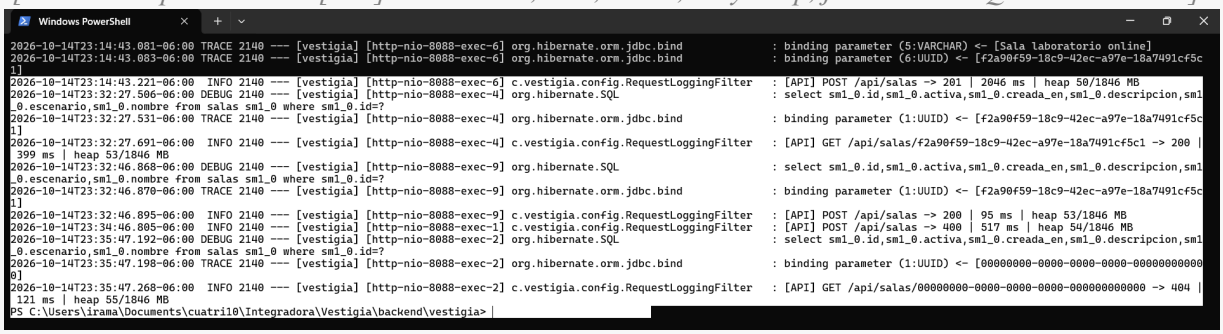


2.8. Logs del servidor Spring Boot

Trazas generadas por el backend durante la recepción del POST y la ejecución de las consultas GET, con el SQL ejecutado, los códigos de respuesta, los tiempos de procesamiento y el consumo de memoria.

Figura 8. Application logs de Spring Boot (GET y POST)

[Insertar captura: líneas [API] con método, ruta, status, ms y heap, junto con el SQL de Hibernate]

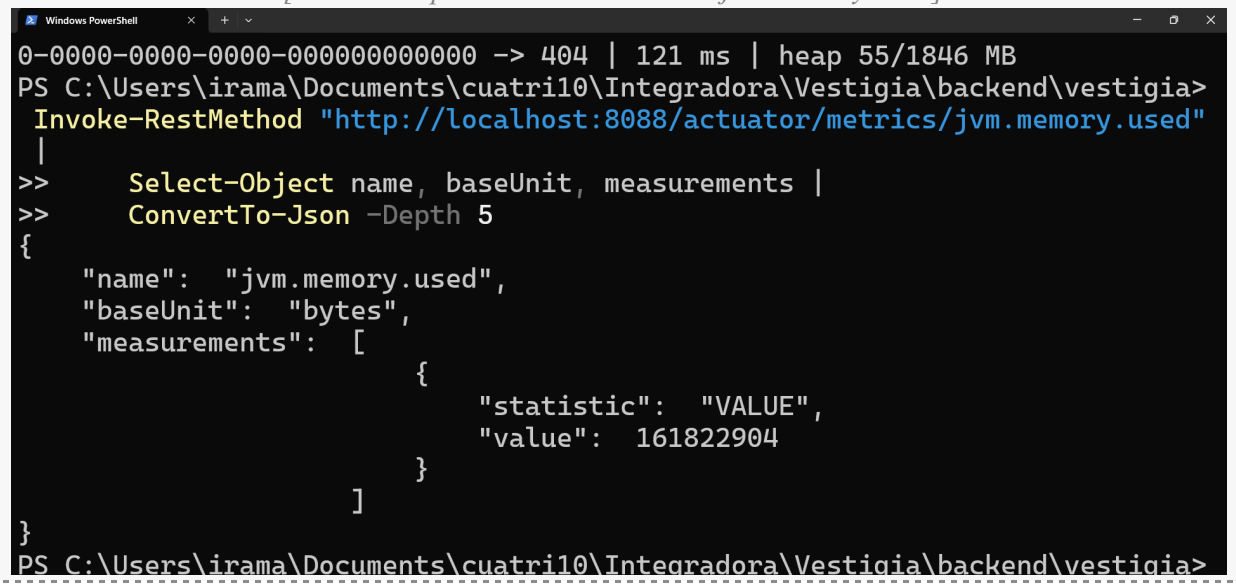


2.9. Métricas de memoria del servidor

Consulta al endpoint de métricas del servidor para documentar el consumo de memoria de la aplicación durante las pruebas.

Figura 9. Métrica jvm.memory.used de Actuator

[Insertar captura: /actuator/metrics/jvm.memory.used]



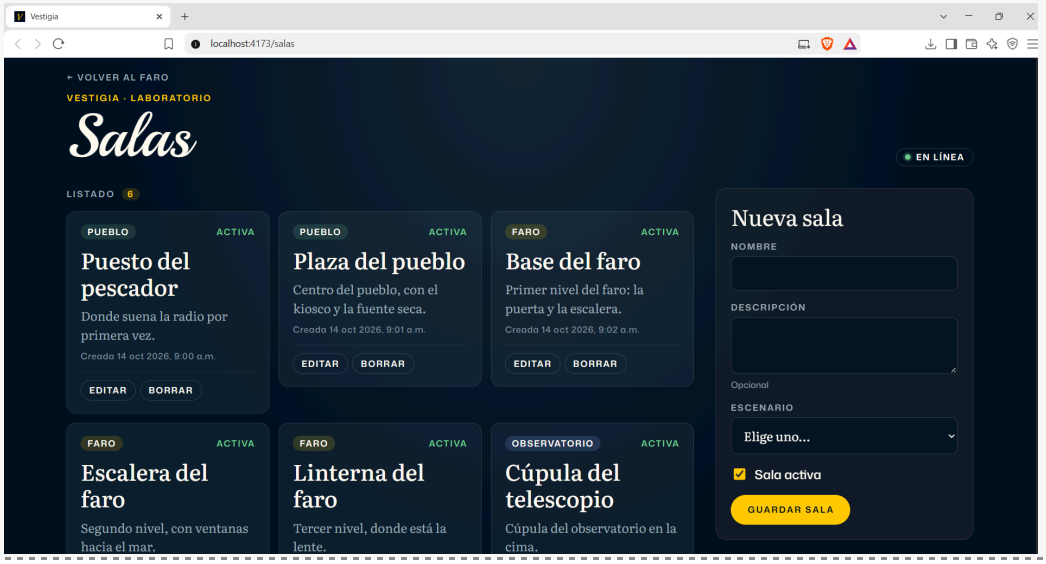
3. Evidencias del frontend y comportamiento PWA

3.1. Listado de datos renderizado

Vista de la PWA con el listado de salas renderizado correctamente tras realizar la consulta GET.

Figura 10. Listado de salas en la PWA

[Insertar captura: pantalla del listado de salas]

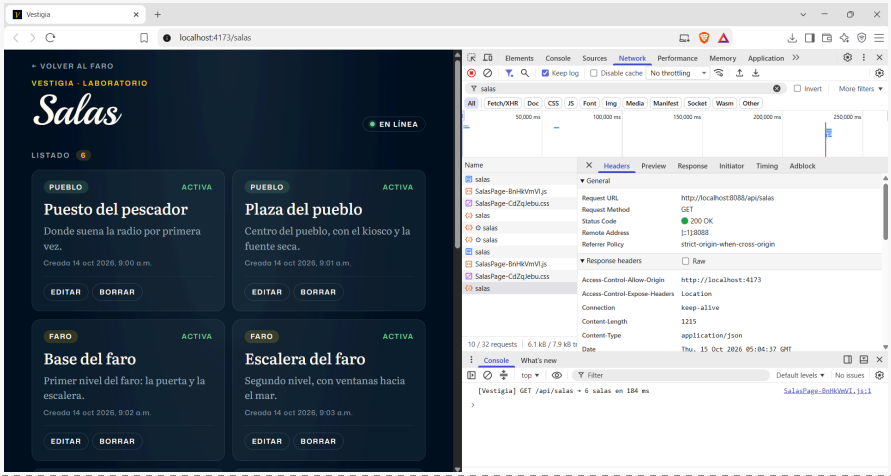


3.2. DevTools: pestaña Network (consulta)

Tráfico de red de la consulta del listado, con los tiempos de respuesta (latencia), el tamaño del payload y los headers HTTP.

Figura 11. Network: petición GET /api/salas

[Insertar captura: Network con tiempos, tamaño y headers del GET]

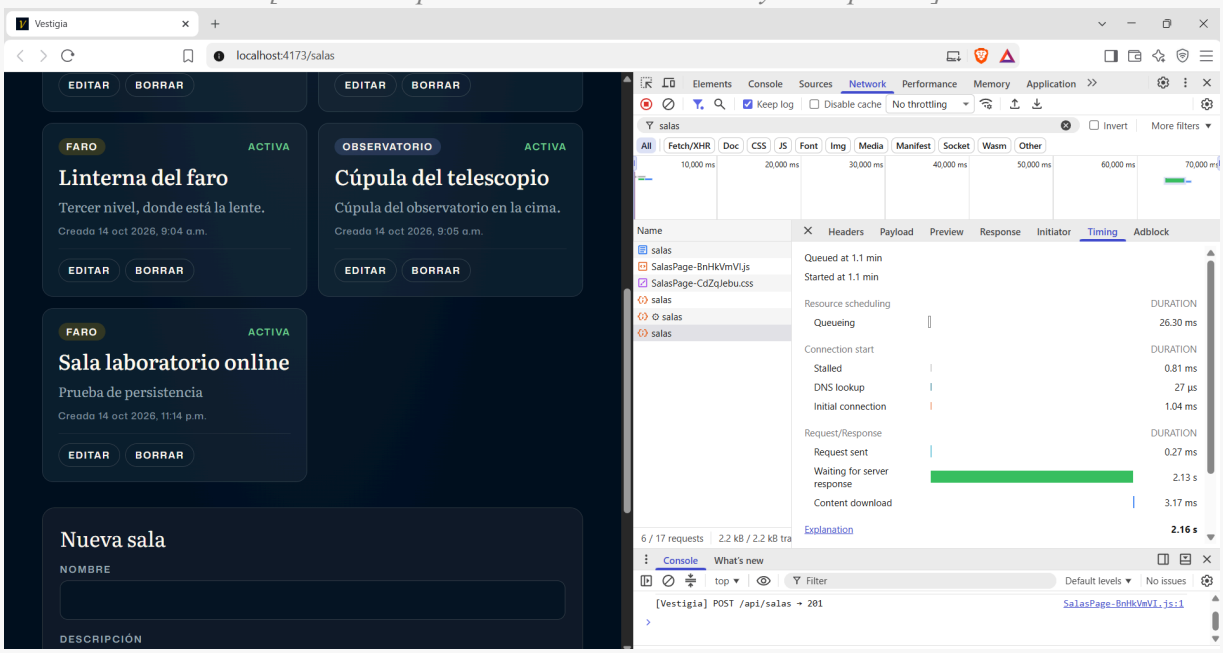


3.3. DevTools: pestaña Network (envío)

Tráfico de red del envío de una nueva sala desde la PWA, con su latencia, payload y headers.

Figura 12. Network: petición POST /api/salas

[Insertar captura: Network con el POST y su respuesta]

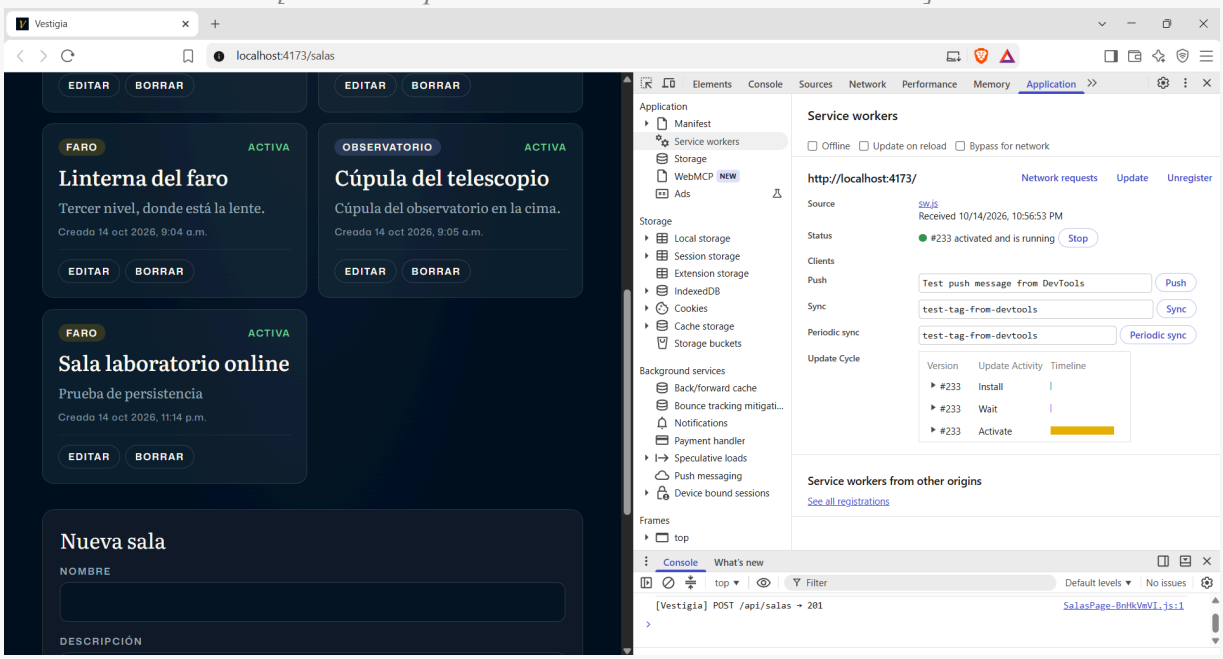


3.4. DevTools: Service Worker activo

Pestaña Application mostrando el Service Worker registrado y activo.

Figura 13. Application: Service Workers

[Insertar captura: Service Worker en estado activated]

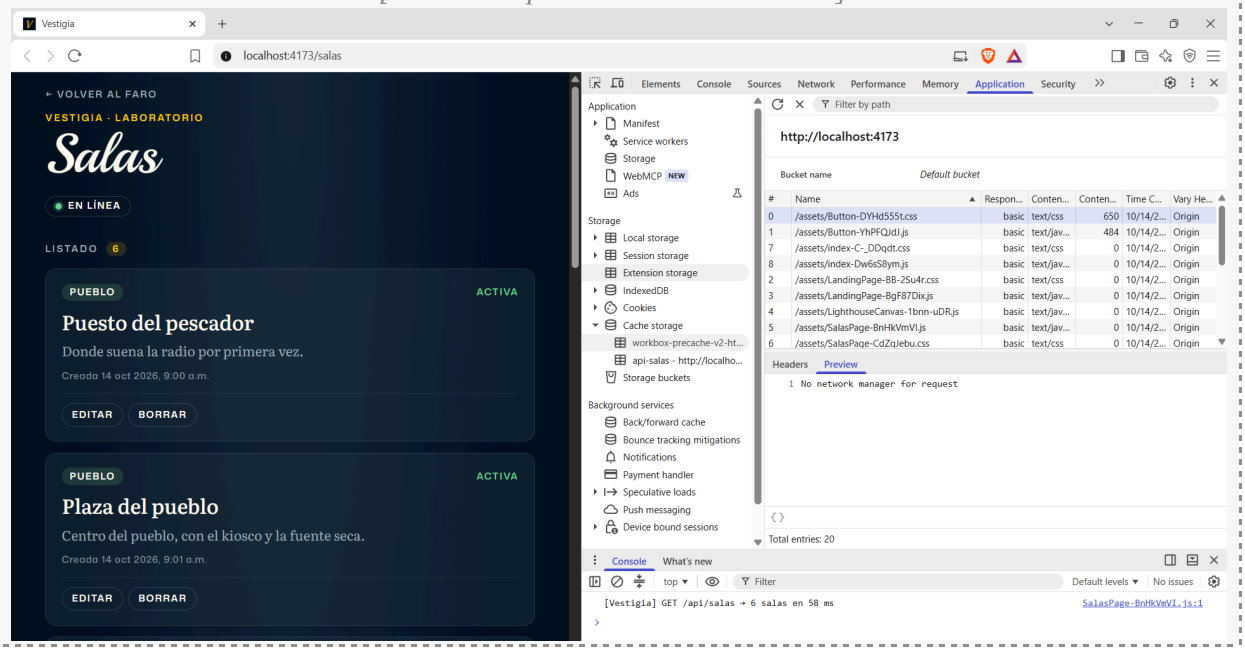


3.5. DevTools: Cache Storage

Estado del almacenamiento en caché con los recursos y las respuestas de la API que se conservan para uso sin conexión.

Figura 14. Application: Cache Storage

[Insertar captura: entradas del caché]

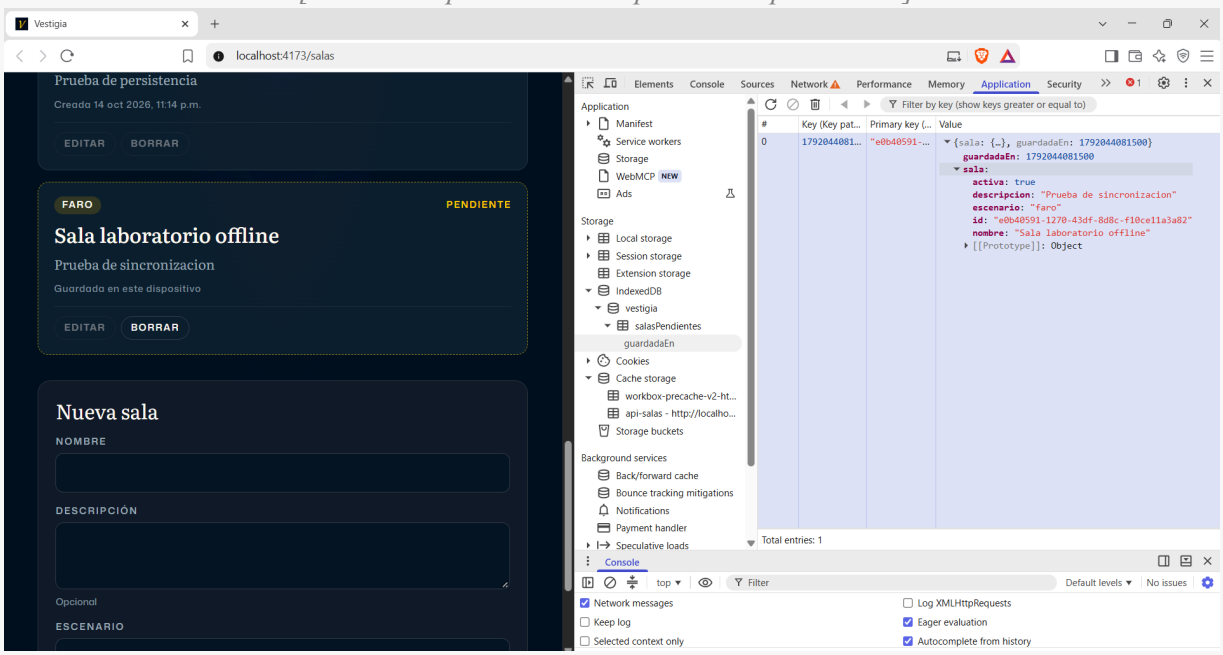


3.6. DevTools: IndexedDB

Estado de IndexedDB con las operaciones pendientes de sincronizar creadas mientras no había conexión.

Figura 15. Application: IndexedDB

[Insertar captura: cola de operaciones pendientes]

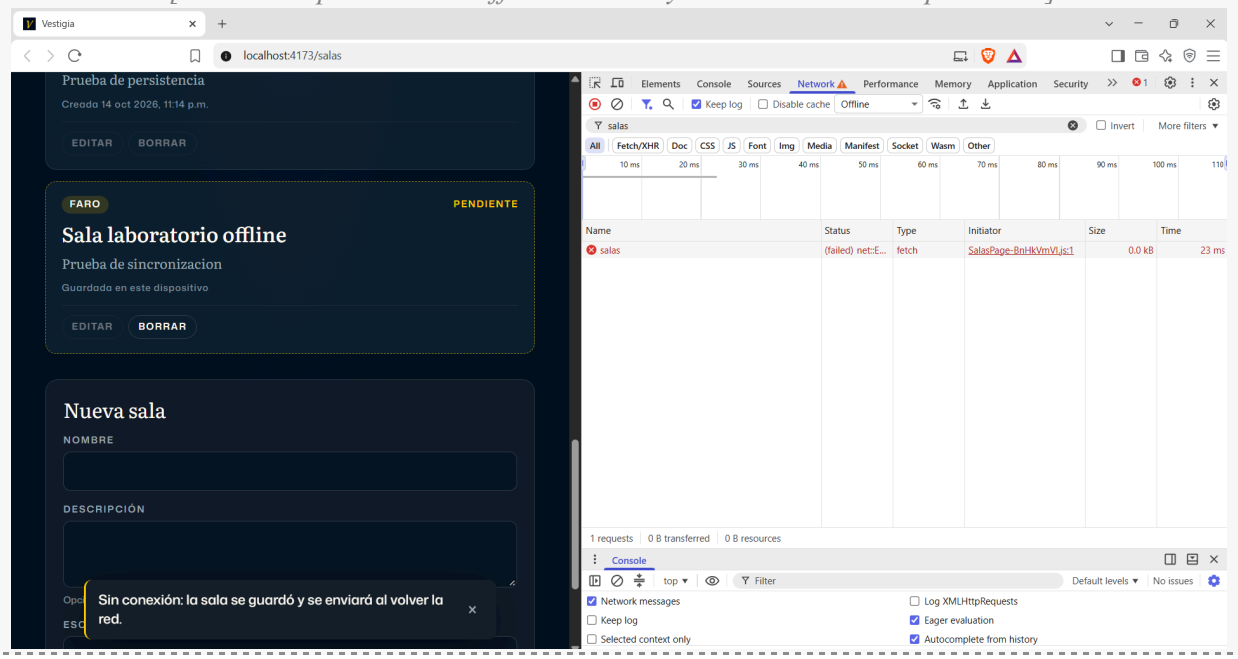


3.7. Comportamiento sin conexión

Creación de una sala con la red desactivada. La PWA conserva la operación, la muestra como pendiente e informa el estado de conexión al usuario.

Figura 16. PWA en modo sin conexión con una sala pendiente

[Insertar captura: modo Offline activado y sala marcada como pendiente]

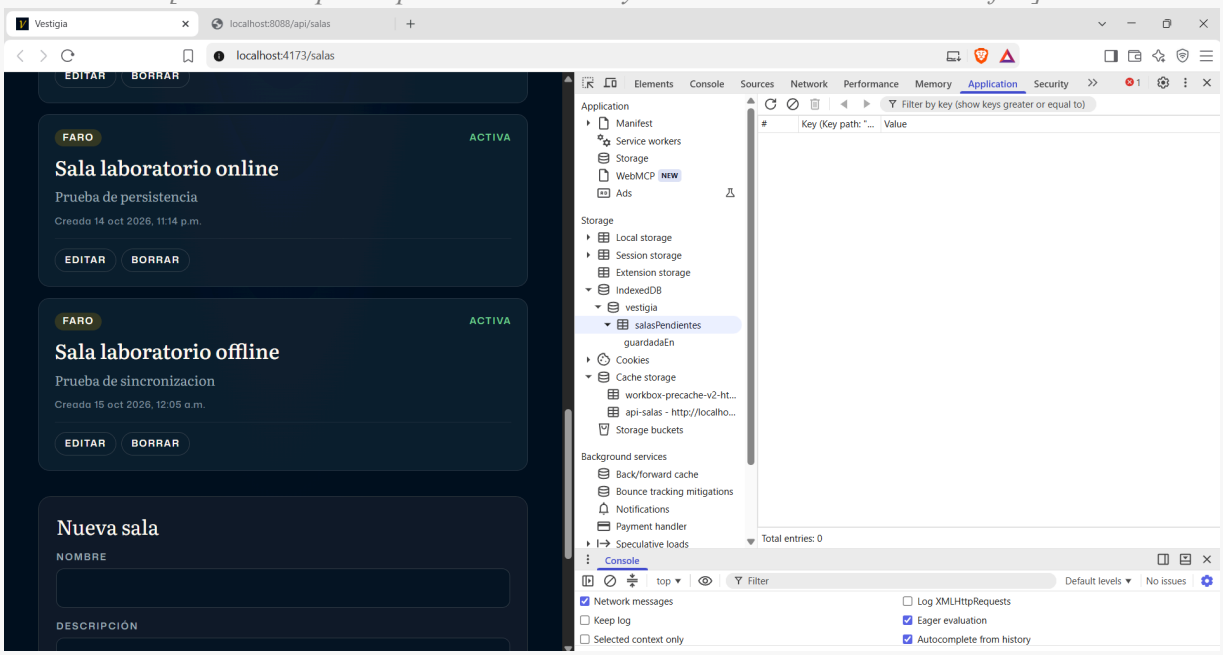


3.8. Sincronización al recuperar la conexión

Al volver la conexión, la PWA reenvía la operación pendiente, el backend la persiste y la interfaz actualiza su estado.

Figura 17. Sincronización online tras el envío offline

[Insertar captura: petición reenviada y estado actualizado en la interfaz]

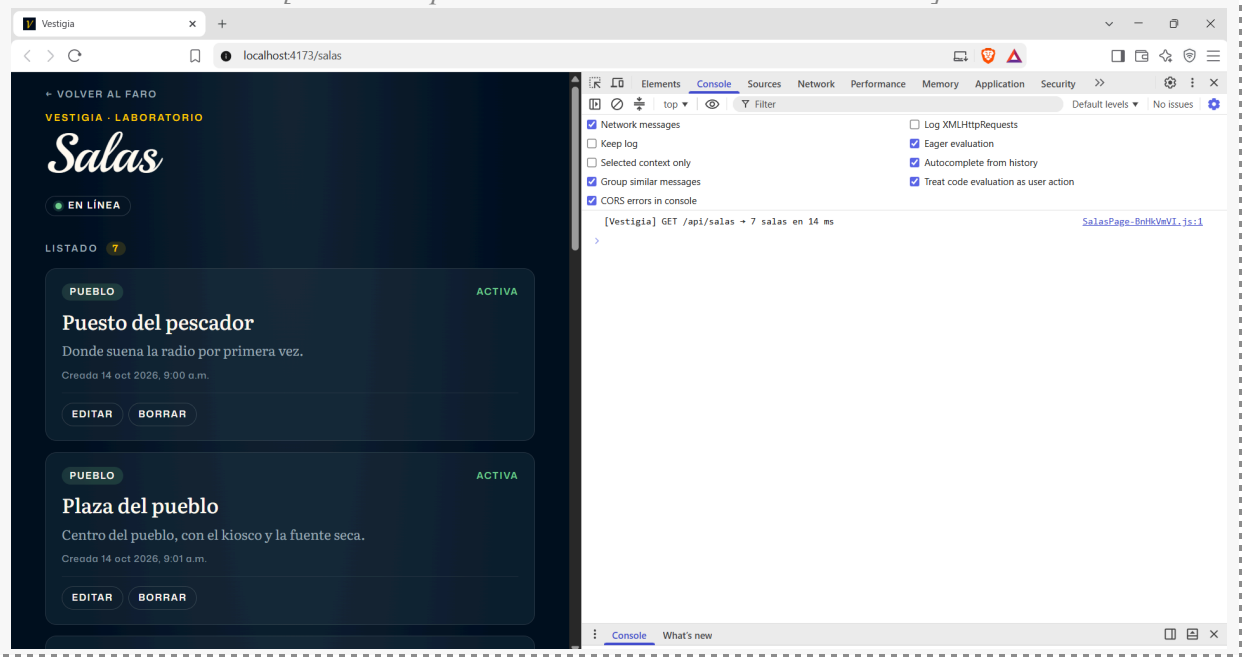


3.9. DevTools: pestaña Console

Consola del navegador durante el consumo y la renderización de los datos.

Figura 18. Console durante la consulta y el renderizado

[Insertar captura: consola sin errores durante el GET]



4. Resumen de casos de prueba

La tabla 1 resume los casos ejecutados, el resultado esperado y la evidencia que los respalda.

Tabla 1. Resumen de casos de prueba

ID	Caso de prueba	Resultado esperado	Resultado obtenido	Figuras
CP-01	Consulta del listado (GET)	Código 200 y arreglo JSON con las salas.	La API respondió con código **200** y un arreglo JSON. Las salas se renderizaron correctamente en la PWA.	3, 10
CP-02	Envío de una sala nueva (POST)	Código 201 y registro persistido en la base de datos.	El POST respondió con código **201**. La nueva sala quedó almacenada en PostgreSQL con el mismo UUID y los datos enviados.	4, 5

ID	Caso de prueba	Resultado esperado	Resultado obtenido	Figuras
CP-03	Reenvío del mismo identificador	Código 200 y una sola fila en la base de datos.	El reenvío respondió con código **200**. La consulta confirmó una sola fila con ese UUID, sin duplicación del registro.	6
CP-04	Datos inválidos	Código 400 con detalle del campo.	La API respondió con código **400**, indicando los errores del nombre vacío y del escenario no permitido.	7
CP-05	Recurso inexistente	Código 404.	La consulta de un UUID inexistente respondió con código **404** y un mensaje indicando que la sala no existe.	7
CP-06	Service Worker y caché	Service Worker activo y recursos en Cache Storage.	El Service Worker se encontró activo y Cache Storage mostró los recursos y las respuestas de la API almacenados.	13, 14
CP-07	Creación sin conexión	Operación en cola (IndexedDB) y marcada como pendiente.	La sala creada sin conexión quedó guardada en IndexedDB y se mostró como pendiente de sincronización en la interfaz.	15, 16
CP-08	Sincronización al reconectar	Operación reenviada y registro único en la base de datos.	Al recuperar la conexión, la operación pendiente se envió al backend y salió de la cola. PostgreSQL conservó un único registro con el mismo UUID.	17, 6

Conclusiones

El laboratorio permitió comprobar la integración entre la PWA de Vestigia, el backend en Spring Boot y PostgreSQL. Se verificó la consulta y renderización de las salas, el registro de nuevos datos y su persistencia en la base de datos. También se comprobó que reenviar una solicitud con el mismo identificador no duplica el registro y que la API responde con los códigos correspondientes ante datos inválidos o recursos inexistentes.

Las evidencias de DevTools, los logs del servidor y las métricas de memoria facilitaron el seguimiento de las operaciones. La revisión del Service Worker, la caché y la cola en IndexedDB complementó la evaluación del comportamiento offline y la sincronización al recuperar la conexión. Estas pruebas proporcionan una base reproducible para detectar fallas de integración y verificar futuras modificaciones del proyecto.

Referencias

Equipo Vestigia. (2026). *Vestigia* [Repositorio de GitHub]. https://github.com/Oliveriswackk/Vestigia

Equipo Vestigia. (2026). *vestigia-pwa* [Repositorio de GitHub]. https://github.com/AiltonMV/vestigia-pwa